

PATRONES DE SOFTWARE
PLATAFORMA DE VOTACIÓN ELECTRONICA

JOHINER MIGUEL RINCON RUEDA
NÉSTOR JULIO LOZANO DÁVILA

GRUPO
E195

DOCENTE
ELIECER MONTERO OJEDA

INSTITUCIÓN UNIVERSITARIA TECNOLÓGICA DE SANTANDER
BUCARAMANGA – SANTANDER
AGOSTO DE 2026

Contenido

Semana 1 15/08:	3
Semana 2 22/08:	4
Semana 4 04/09:	14
Semana5 12/09	22

Semana 1 15/08:

Asignación de proyecto

Plataforma de votación electrónica

- Sistema de autenticación segura de votantes
- Conteo automatizado con verificación
- Auditoria completa del proceso
- Escalabilidad para elecciones nacionales

Contextualización

Descripción del proyecto

El proyecto consiste en realizar una plataforma de votación electrónica, donde los usuarios puedan realizar su voto de una manera segura y sencilla. La idea es que el sistema permita controlar todo el proceso de votación, desde el ingreso y autenticación de los votantes, hasta el conteo de los votos y la revisión de los resultados.

También se quiere que la plataforma tenga un sistema de auditoría, para poder revisar las acciones importantes que se realizaron durante el proceso y tener un mayor control sobre la información.

Se busca aplicar algunos patrones que nos ayuden a organizar mejor el sistema y que en un futuro pueda ser más fácil de mantener y ampliar. Además, se tiene en cuenta que el sistema pueda soportar una cantidad grande de usuarios, pensando en un escenario donde pueda utilizarse para elecciones a nivel nacional.

Objetivo general

Desarrollar el diseño de una plataforma de votación electrónica que permita realizar el proceso de votación de una forma segura y organizada, teniendo en cuenta la autenticación de los votantes, el conteo de los votos, la auditoría del proceso y la posibilidad de que el sistema pueda crecer para soportar una gran cantidad de usuarios.

Objetivos específicos

- Diseñar un sistema de autenticación que permita verificar a los votantes antes de realizar su voto.
- Implementar un proceso para registrar y realizar el conteo automático de los votos, buscando que los resultados sean confiables.
- Crear un mecanismo de auditoría que permita revisar las acciones importantes realizadas durante el proceso de votación.
- Aplicar patrones de diseño de software para organizar mejor la plataforma y facilitar su mantenimiento y crecimiento.

Semana 2 22/08:

Primer patrón aplicado — Singleton

Empezamos a implementar el primer patrón sobre la base que dejamos en la semana 1. revisamos el código inicial del sistema y notamos que la configuración, el logging y las conexiones a base de datos estaban repartidos por todos lados sin ningún control, cada clase con sus propios valores hard-codeados. Decidimos aplicar Singleton en tres partes puntuales: ConfiguracionGlobal, LoggerCentralizado y PoolConexionesDB.

La razón principal es que en una plataforma de votación no puede haber "varias versiones" corriendo de estas tres cosas al mismo tiempo. Si dos módulos leyeran configuraciones distintas, o si hubiera más de un logger escribiendo por separado, se pierde el control sobre lo que realmente pasó durante el proceso, y eso justo choca con lo que pedía el proyecto desde el principio en la parte de auditoría.

1. Parte del código donde se evidencia el patrón

En el desarrollo de SecureBallot, aplicamos el patrón Singleton en tres componentes clave: ConfiguracionGlobal, LoggerCentralizado y PoolConexionesDB. En JavaScript

implementamos los tres ingredientes clásicos de la teoría orientada a objetos (adaptados mediante la sintaxis moderna con #):

- Un campo estático privado (static #instancia = null) que almacena la única instancia existente.
- Un "constructor privado" simulado que evalúa si la instancia ya existe y lanza un error (throw new Error(...)) para bloquear el uso de new.
- Un método estático público (obtenerInstancia()) que actúa como la única puerta de entrada permitida (aplicando lazy initialization).

→ Configuración Global (Singleton)

Evidencia Implementación del patrón en el código

```
// =====  
// 1. CONFIGURACIÓN GLOBAL (Singleton)  
// =====  
  
// Guarda credenciales y parámetros del sistema en un solo lugar, para que  
// ningún módulo (auth, conteo, auditoría) trabaje con una copia distinta  
// o desactualizada de la configuración.  
// =====  
class ConfiguracionGlobal {  
  // [INGREDIENTE 1] Campo static privado: aquí y solo aquí vive la  
  // única instancia de esta clase que va a existir en todo el sistema.  
  static #instancia = null;  
  
  #config;  
  
  constructor() {  
    // [INGREDIENTE 2] "Constructor privado" simulado: si ya existe una  
    // instancia y alguien intenta crear otra con "new", se bloquea aquí.  
    if (ConfiguracionGlobal.#instancia) {  
      throw new Error('Usa ConfiguracionGlobal.obtenerInstancia(), no  
"new"');  
    }  
  
    // Se arma la configuración leyendo variables de entorno (nunca
```

```

// credenciales escritas directamente en el código).
this.#config = {
  dbHost: process.env.DB_HOST,
  dbPort: parseInt(process.env.DB_PORT || '5432', 10),
  dbUser: process.env.DB_USER,
  dbPass: process.env.DB_PASS,
  apiKey: process.env.API_KEY,
  maxConexiones: parseInt(process.env.MAX_CONEXIONES || '100', 10),
  timeout: parseInt(process.env.TIMEOUT || '30000', 10),
  smtpHost: process.env.SMTP_HOST,
  smtpPort: parseInt(process.env.SMTP_PORT || '587', 10),
  smtpUser: process.env.SMTP_USER,
  smtpPass: process.env.SMTP_PASS,
};

this.#validarConfiguracionCritica();
ConfiguracionGlobal.#instancia = this;
}

#validarConfiguracionCritica() {
  // Falla rápido al arrancar si falta algo esencial, en vez de fallar
  // silenciosamente a mitad de una elección.
  const requeridas = ['dbHost', 'dbUser', 'dbPass', 'apiKey'];
  const faltantes = requeridas.filter((c) => !this.#config[c]);
  if (faltantes.length > 0) {
    throw new Error(
      `Faltan variables de entorno requeridas: ${faltantes.join(', ')}`
    );
  }
}

// [INGREDIENTE 3] Método static público: única puerta de entrada.
// Si no existe la instancia la crea (lazy initialization), si ya
// existe simplemente devuelve la misma de siempre.
static obtenerInstancia() {
  if (!ConfiguracionGlobal.#instancia) {
    new ConfiguracionGlobal();
  }
  return ConfiguracionGlobal.#instancia;
}

get(clave) {
  return this.#config[clave];
}

```

```

urlConexionDB() {
  const c = this.#config;
  return `jdbc:postgresql://${c.dbHost}:${c.dbPort}/sistema_votacion`;
}
}

```

→2- Logger Centralizado (Singleton)

Evidencia implementación del patron en el código

```

// =====
// 2. LOGGER CENTRALIZADO (Singleton) – con cadena de hashes
// =====

// Este es el más importante para la auditoría del proyecto: si hubiera
// varias instancias del logger, el historial de eventos quedaría
// partido entre ellas y ya no se podría confiar en el orden real de
// lo que pasó durante la votación.
// =====

class LoggerCentralizado {

  // [INGREDIENTE 1] Campo static privado.

  static #instancia = null;

  #historial = [];
  // Hash "semilla" (64 ceros) que arranca la cadena. El primer registro
  // se encadena contra este valor.
  #ultimoHash = '0'.repeat(64);

  constructor() {
    // [INGREDIENTE 2] "Constructor privado" simulado.
    if (LoggerCentralizado.#instancia) {
      throw new Error('Usa LoggerCentralizado.obtenerInstancia(), no
"new"');
    }
    LoggerCentralizado.#instancia = this;
  }

  // [INGREDIENTE 3] Método static público.
  static obtenerInstancia() {

```

```

    if (!LoggerCentralizado.#instancia) {
        new LoggerCentralizado();
    }
    return LoggerCentralizado.#instancia;
}

// Genera un hash SHA-256 del objeto recibido. Se usa para encadenar
// cada entrada del log con la anterior.
#hashDe(objeto) {
    return
    crypto.createHash('sha256').update(JSON.stringify(objeto)).digest('hex');
}

registrar(nivel, mensaje, contexto = {}) {
    const entrada = {
        timestamp: new Date().toISOString(),
        nivel,
        mensaje,
        contexto,
        hashAnterior: this.#ultimoHash, // encadena esta entrada con la
anterior
    };
    entrada.hash = this.#hashDe(entrada);
    this.#ultimoHash = entrada.hash;

    // Object.freeze: una vez guardada, la entrada no se puede modificar
    // en memoria. Es una capa extra de protección del rastro de auditoría.
    this.#historial.push(Object.freeze(entrada));

    console.log(`[${entrada.timestamp}] [${nivel}] ${mensaje}`);
    return entrada.hash;
}

// Método específico para eventos de voto: nunca se registra la
// identidad del votante, solo el evento y un id anónimo/hash. Esto es
// clave para no comprometer el secreto del voto mientras se audita.
registrarVoto(idVotoAnonimo, mesaOColegio) {
    return this.registrar('AUDITORIA_VOTO', 'Voto registrado', {
        idVotoAnonimo,
        mesaOColegio,
    });
}

obtenerHistorial() {

```

```

    return [...this.#historial]; // copia defensiva, el original es
    immutable
  }

  // Recalcula la cadena de hashes desde cero y compara contra la
  // guardada: si alguien alteró una entrada (en memoria o en la BD),
  // el hash recalculado no va a coincidir y esto devuelve false.
  // Esto es la "prueba" de que se puede correr para el Punto 3.
  verificarIntegridad() {
    let hashPrevio = '0'.repeat(64);
    for (const entrada of this.#historial) {
      const { hash, ...resto } = entrada;
      const hashEsperado = this.#hashDe({ ...resto, hashAnterior: hashPrevio
    });
      if (hashEsperado !== hash) return false;
      hashPrevio = hash;
    }
    return true;
  }
}

```

→3- Pool de conexiones a BD (Singleton)

Evidencia implementación del patron en el código

```

// =====
// 3. POOL DE CONEXIONES A BD (Singleton)
// =====

// Evita que el sistema abra conexiones a la base de datos sin control,
// algo crítico pensando en un escenario de elección nacional con miles
// de personas votando al mismo tiempo.
// =====
class PoolConexionesDB {
  // [INGREDIENTE 1] Campo static privado.
  static #instancia = null;

  #conexionesActivas = 0;
  #maxConexiones;
  #colaEspera = []; // peticiones que esperan turno cuando se llega al
  límite

  constructor() {
    // [INGREDIENTE 2] "Constructor privado" simulado.
  }
}

```

```

    if (PoolConexionesDB.#instancia) {
        throw new Error('Usa PoolConexionesDB.obtenerInstancia(), no "new"');
    }
    // Nota: este Singleton depende de otro Singleton (ConfiguracionGlobal)
    // para saber cuál es el máximo de conexiones permitido.
    const config = ConfiguracionGlobal.obtenerInstancia();
    this.#maxConexiones = config.get('maxConexiones');
    PoolConexionesDB.#instancia = this;
}

// [INGREDIENTE 3] Método static público.
static obtenerInstancia() {
    if (!PoolConexionesDB.#instancia) {
        new PoolConexionesDB();
    }
    return PoolConexionesDB.#instancia;
}

async obtenerConexion() {
    const logger = LoggerCentralizado.obtenerInstancia();

    // Si ya se llegó al máximo de conexiones simultáneas, la petición
    // se queda esperando en una cola en vez de saturar la base de datos.
    if (this.#conexionesActivas >= this.#maxConexiones) {
        logger.registrar('WARN', 'Límite de conexiones alcanzado, solicitud en cola');
        await new Promise((resolve) => this.#colaEspera.push(resolve));
    }

    this.#conexionesActivas++;
    logger.registrar(
        'DEBUG',
        `Conexión adquirida
    (${this.#conexionesActivas}/${this.#maxConexiones})`
    );

    return {
        id: crypto.randomUUID(),
        liberar: () => this.#liberarConexion(),
    };
}

#liberarConexion() {
    this.#conexionesActivas--;
    // Si hay alguien esperando en la cola, le da paso apenas se libera

```

```
// una conexión.  
const siguiente = this.#colaEspera.shift();  
if (siguiente) siguiente();  
}  
  
get conexionesActivas() {  
  return this.#conexionesActivas;  
}  
}
```

2- Interpretacion y aporte del patron del proyecto

Para SecureBallot apliqué Singleton en tres partes: ConfiguracionGlobal, LoggerCentralizado y PoolConexionesDB. Elegí estas tres porque en un sistema de votación no puede haber "varias versiones" de ellas funcionando a la vez, eso genera problemas.

ConfiguracionGlobal es la más simple de explicar. Si cada parte del sistema (login, conteo, auditoría) tuviera su propia copia de la config —host de la BD, api keys, etc—, en algún momento una copia queda vieja y ya empieza el desorden, porque un módulo estaría usando datos distintos a otro sin que nadie se dé cuenta. Con Singleton hay una sola copia y todos leen de ahí.

Con LoggerCentralizado la cosa es más delicada. Una plataforma de votación necesita poder mostrar qué pasó durante el proceso si alguien lo pide. Si hubiera varios loggers al mismo tiempo, el historial quedaría partido en pedazos y ya no se podría confiar en el orden real de los eventos. Por eso además de ser Singleton, cada registro guarda el hash del registro anterior (hashAnterior -> hash), entonces si alguien edita algo después la cadena deja de cuadrar y verificarIntegridad() lo agarra. Esto es literal lo que pide el proyecto en la parte de auditoría.

Y PoolConexionesDB, pensando en que esto en teoría podría usarse para una elección nacional con un montón de gente votando a la vez: si cada petición abriera su propia conexión a la BD sin control,

se puede saturar la base de datos y ahí sí se cae todo justo cuando más se necesita. El pool lleva la cuenta de cuántas conexiones hay activas y pone en cola las que sobran.

Con SOLID: por un lado cumple Responsabilidad Única porque cada clase además de hacer lo suyo también controla su propia creación (el `#instancia` y el bloqueo en el constructor). Por otro lado, hay que admitir que rompe un poco Inversión de Dependencias, porque llamar `obtenerInstancia()` desde cualquier lado del código te acopla directamente a esa clase en vez de depender de una abstracción. Aun así, para estos tres casos me pareció que valía la pena ese acoplamiento a cambio de tener una única fuente de verdad.

Evidencia: código en `singleton-votacion.js`, pruebas en `tests/singleton.test.js`.

3. Ejecución de pruebas y resultados

El comportamiento del patrón fue verificado mediante pruebas automatizadas con Jest, confirmando que las tres clases devuelven siempre la misma instancia, que el uso directo de `new` lanza un error en todas ellas, que la cadena de hashes mantiene su integridad y que el pool de conexiones regula eficientemente la concurrencia.

Evidencia pruebas con la ejecución de Jest

```
at LoggerCentralizado.log [as registrar] (Patrones/01-Singleton/singleton-votacion.js:162:13)
```

PASS Patrones/01-Singleton/__tests__/singleton.test.js

ConfiguracionGlobal

✓ obtenerInstancia() siempre devuelve el mismo objeto (3 ms)

✓ usar "new" directamente lanza un error (18 ms)

LoggerCentralizado

✓ obtenerInstancia() siempre devuelve el mismo objeto (2 ms)

✓ la cadena de hashes mantiene su integridad tras registrar un voto (12 ms)

PoolConexionesDB

✓ obtenerInstancia() siempre devuelve el mismo objeto

✓ usar "new" directamente lanza un error (1 ms)

✓ el contador de conexiones activas sube al pedir una conexion (5 ms)

Test Suites: 1 passed, 1 total

Tests: 7 passed, 7 total

Snapshots: 0 total

Time: 0.402 s, estimated 1 s

Ran all test suites.

PS C:\Users\johin\SecureBallot>

Semana 4 04/09:

FASE 1 - Patrón Factory Method

Durante la Semana 4 se trabajó con el patrón de diseño Factory Method, continuando con el desarrollo de la plataforma de votación electrónica SecureBallot.

En esta sesión se estudió cómo este patrón permite delegar la creación de objetos, evitando que la lógica principal del sistema dependa directamente de clases concretas. También se analizó su relación con los principios SOLID, principalmente el Principio de Abierto/Cerrado (OCP), buscando que el sistema pueda extenderse con nuevos tipos de objetos sin modificar el código que ya funciona.

1. Contextualización

En las sesiones anteriores se trabajó el patrón Singleton, utilizado en SecureBallot para controlar la existencia de una única instancia de componentes como ConfiguracionGlobal, LoggerCentralizado y PoolConexionesDB.

Para esta semana se buscó aplicar un patrón diferente, enfocado en el problema de la creación de objetos.

En un sistema como SecureBallot pueden existir diferentes formas de generar información a partir de los resultados electorales. Por ejemplo, los resultados pueden necesitar ser presentados como un reporte PDF, un archivo CSV o posteriormente en otros formatos.

Una implementación directa mediante múltiples condiciones podría generar un código similar a:

```
if (formato === 'PDF') {  
    reporte = new ReporteResultadosPDF();  
} else if (formato === 'CSV') {  
    reporte = new ReporteResultadosCSV();  
}
```

Este enfoque hace que la lógica principal conozca directamente las clases concretas. Además, cada vez que se agregue un nuevo formato sería necesario modificar el código existente.

Por esta razón se decidió utilizar Factory Method para separar la creación de los reportes de la lógica encargada de utilizarlos.

2. Problema identificado

El problema principal identificado fue el acoplamiento entre la lógica de generación de reportes y las clases concretas de cada formato.

Si la plataforma necesitara soportar nuevos formatos, una implementación basada en if o switch tendría que modificarse constantemente. Esto aumenta la posibilidad de errores y dificulta el mantenimiento del sistema.

Con Factory Method se busca que el código que genera el reporte trabaje con una abstracción común y que la decisión sobre qué objeto concreto crear sea delegada a los creadores correspondientes.

3. Patrón aplicado

Se implementó el patrón Factory Method para la generación de reportes de resultados electorales.

La estructura utilizada permite manejar diferentes formatos mediante productos concretos y creadores concretos.

El producto (ReporteResultados) define el comportamiento común que deben cumplir los diferentes tipos de reportes.

Los productos concretos implementan la generación específica de cada formato.

Por otra parte, GeneradorReporte representa el creador y contiene el método de fábrica encargado de crear el producto. Los creadores concretos determinan qué tipo de reporte se debe generar.

4. Aplicación en SecureBallot

La aplicación del patrón se realizó sobre la generación de reportes de resultados electorales.

Inicialmente se implementaron los formatos:

- ReporteResultadosPDF
- ReporteResultadosCSV

Posteriormente se agregó:

- ReporteResultadosJSON

El nuevo formato se utilizó como evidencia de aplicación del principio OCP, debido a que se incorporó como una extensión sin modificar las clases existentes encargadas de los formatos anteriores.

Además, la generación de los reportes se integró con el LoggerCentralizado desarrollado anteriormente con Singleton. De esta manera, los eventos importantes relacionados con la generación de reportes continúan registrándose mediante el mismo mecanismo de auditoría del sistema.

5. Relación con SOLID

La implementación del patrón presenta relación principalmente con los siguientes principios:

OCP (Open/Closed Principle):

El sistema puede extenderse mediante nuevos tipos de reportes sin modificar las clases existentes. La incorporación del formato JSON permitió comprobar esta característica.

SRP (Single Responsibility Principle):

La responsabilidad de generar cada formato se encuentra separada de la lógica general de generación de reportes. Los productos concretos se encargan de su formato específico, mientras que el creador se encarga de coordinar el proceso.

DIP (Dependency Inversion Principle):

La lógica general trabaja con la abstracción `ReporteResultados` en lugar de depender directamente de cada implementación concreta.

LSP (Liskov Substitution Principle):

Los diferentes reportes concretos pueden utilizarse como implementaciones de `ReporteResultados`, siempre que respeten el contrato definido por la abstracción.

6. Evidencia de implementación

La implementación del patrón se encuentra organizada dentro de la carpeta:

`Semana04_PFM/`

Entre los componentes desarrollados se encuentran:

- `reporte-resultados.js`
- `generador-reporte.js`
- `reporte-json-extension.js`
- `factory-method.test.js`
- `index.js`

El archivo de extensión `reporte-json-extension.js` se utilizó específicamente para demostrar que es posible incorporar un nuevo producto mediante extensión, sin modificar las implementaciones anteriores.

7. Evidencia de pruebas

Para verificar el funcionamiento de la implementación se utilizaron pruebas automatizadas mediante Jest.

Se ejecutó el siguiente comando:

`npx jest`

El resultado obtenido fue:

```
at LoggerCentralizado.log [as registrar] (Semana03/singleton-votacion.js:162:13)
console.log
[2026-09-05T02:29:38.291Z] [INFO] Reporte de resultados electorales generado
at LoggerCentralizado.log [as registrar] (Semana03/singleton-votacion.js:162:13)
console.log
[2026-09-05T02:29:38.293Z] [INFO] Reporte de resultados electorales generado
at LoggerCentralizado.log [as registrar] (Semana03/singleton-votacion.js:162:13)
PASS Semana03/___tests___/singleton.test.js
  ● Console

  console.log
    [2026-09-05T02:29:38.401Z] [AUDITORIA_VOTO] Voto registrado
    at LoggerCentralizado.log [as registrar] (Semana03/singleton-votacion.js:162:13)

  console.log
    [2026-09-05T02:29:38.407Z] [DEBUG] Conexión adquirida (1/100)
    at LoggerCentralizado.log [as registrar] (Semana03/singleton-votacion.js:162:13)

Test Suites: 2 passed, 2 total
Tests:       16 passed, 16 total
Snapshots:   0 total
Time:        0.867 s, estimated 1 s
Ran all test suites.
PS E:\Descargas\Patrones\SecureBallot>
```

Las dos suites correspondientes al proyecto fueron aprobadas:

PASS Semana04_PFM/factory-method.test.js

PASS Semana03/___tests___/singleton.test.js

En total se aprobaron 16 pruebas de 16, sin pruebas fallidas.

Las pruebas permitieron comprobar tanto el funcionamiento de Factory Method como la continuidad del funcionamiento del Singleton desarrollado anteriormente.

Durante la ejecución también se evidenció el registro de eventos mediante el LoggerCentralizado:

[INFO] Reporte de resultados electorales generado

Esto permite comprobar que la nueva implementación utiliza el mecanismo de auditoría existente en SecureBallot.

8. Aporte del patrón al proyecto

La implementación de Factory Method permite que SecureBallot tenga una estructura más flexible para la generación de reportes. La lógica principal no necesita conocer directamente cada formato existente, ya que la creación de los objetos se delega en los creadores concretos.

Uno de los principales aportes es la facilidad para incorporar nuevos formatos. Por ejemplo, después de implementar PDF y CSV se pudo agregar JSON mediante nuevas clases, manteniendo las implementaciones anteriores sin modificaciones.

De esta manera, el patrón contribuye a:

- Reducir el acoplamiento.
- Facilitar la extensión del sistema.
- Separar la creación de objetos de la lógica de negocio.
- Facilitar el mantenimiento.
- Mejorar la organización del código.
- Aplicar el principio OCP de SOLID.

9. Resultado de la Semana 4 fase 1

Al finalizar la semana se logró implementar y probar el patrón Factory Method dentro de SecureBallot, utilizando como caso de aplicación la generación de reportes de resultados electorales.

La implementación permitió demostrar la diferencia entre crear directamente objetos concretos y delegar su creación mediante un método de fábrica.

Además, la incorporación de un nuevo formato de reporte permitió utilizar el proyecto como evidencia práctica del principio Abierto/Cerrado, demostrando que el sistema puede ampliarse sin necesidad de modificar las implementaciones existentes.

La ejecución de las pruebas automatizadas confirmó el correcto funcionamiento de la implementación, obteniendo 16 pruebas aprobadas de 16 y manteniendo funcional el patrón Singleton desarrollado anteriormente.

Evidencias:

▼	📁	Semana04_PFM	●	9
	⚠️	factory-method.test.js	U	10
	JS	generador-reporte.js	U	11
				12
	JS	index.js	U	13
				14
	JS	reporte-json-extension.js	U	15
	JS	reporte-resultados.js	U	16

Ilustración 1 Estructura de archivos de Semana04

```
9
10 const { LoggerCentralizado } = require('../Semana03/singleton-votacion');
11 const { ReporteResultadosPDF, ReporteResultadosCSV } = require('./reporte-resultados');
12
13 > /** ...
25 class GeneradorReporte {
26   // Factory Method: decide QUÉ producto concreto crear. La clase base
27   // no lo implementa; obliga a cada creador concreto a hacerlo.
28   crearReporte() {
29     throw new Error('crearReporte() debe ser implementado por el creador concreto');
30   }
31
32   // Lógica de negocio compartida por TODOS los formatos de reporte.
33   // Si mañana se agrega un paso común (ej. firmar digitalmente el
34   // reporte), se agrega aquí UNA sola vez y todos los formatos lo
35   // heredan automáticamente.
36   generarReporteOficial(datosEleccion) {
37     this.#validar(datosEleccion);
38
39     const reporte = this.crearReporte(); // <- aquí ocurre el Factory Method
40     const resultado = reporte.generar(datosEleccion);
41
42     LoggerCentralizado.obtenerInstancia().registrar(
43       'INFO',
44       'Reporte de resultados electorales generado',
45       { formato: resultado.formato, eleccion: datosEleccion.nombreEleccion }
46     );
47
48     return resultado;
49   }
50 }
```

Ilustración 2 Implementación de GeneradorReporte y los creadores concretos - 1

```

51 #validar(datosEleccion) {
52   if (!datosEleccion || !Array.isArray(datosEleccion.resultados)) {
53     throw new Error('Datos de elección inválidos');
54   }
55 }
56 }
57
58 /**
59  * -----
60  * [INGREDIENTE 4] CREADORES CONCRETOS
61  * -----
62  * Cada uno sobrescribe ÚNICAMENTE crearReporte(). No duplican la
63  * lógica de negocio de generarReporteOficial().
64  */
65 class GeneradorReportePDF extends GeneradorReporte {
66   crearReporte() {
67     return new ReporteResultadosPDF();
68   }
69 }
70
71 class GeneradorReporteCSV extends GeneradorReporte {
72   crearReporte() {
73     return new ReporteResultadosCSV();
74   }
75 }
76
77 module.exports = {
78   GeneradorReporte,
79   GeneradorReportePDF,
80   GeneradorReporteCSV,
81 };
82

```

Ilustración 3 Implementación de GeneradorReporte y los creadores concretos - 2

```

1 > /** ...
17
18 const { ReporteResultados } = require('./reporte-resultados');
19 const { GeneradorReporte } = require('./generador-reporter');
20
21 class ReporteResultadosJSON extends ReporteResultados {
22   generar(datosEleccion) {
23     if (!datosEleccion || !Array.isArray(datosEleccion.resultados)) {
24       throw new Error('Datos de elección inválidos para generar el reporte');
25     }
26     const totalVotos = datosEleccion.resultados.reduce((acc, r) => acc + r.votos, 0);
27
28     const contenido = JSON.stringify(
29       { ...datosEleccion, totalVotos },
30       null,
31       2
32     );
33
34     return { formato: 'json', contenido };
35   }
36
37   extension() {
38     return 'json';
39   }
40 }
41
42 class GeneradorReporteJSON extends GeneradorReporte {
43   crearReporte() {
44     return new ReporteResultadosJSON();
45   }
46 }
47
48 module.exports = {
49   ReporteResultadosJSON,
50   GeneradorReporteJSON,
51 };
52

```

Ilustración 4 Implementación del nuevo formato JSON como evidencia de OCP.

Durante esta semana se trabajó con el patrón de diseño Builder, continuando con el desarrollo de la plataforma de votación electrónica SecureBallot.

En esta sesión se estudió cómo este patrón permite separar la construcción de un objeto complejo de su representación final, permitiendo que el mismo proceso de construcción pueda generar diferentes configuraciones del objeto. También se analizó su relación con los principios SOLID, principalmente el Principio de Inversión de Dependencias (DIP).

1. Contextualización

En las sesiones anteriores se trabajaron los patrones Singleton y Factory Method. Singleton se utilizó para controlar la existencia de una única instancia de componentes críticos (ConfiguracionGlobal, LoggerCentralizado, PoolConexionesDB), y Factory Method se aplicó para delegar la creación de reportes de resultados en distintos formatos (PDF, CSV, JSON).

Para esta semana se buscó aplicar un patrón enfocado en un problema distinto: la construcción de objetos que requieren múltiples parámetros, varios de ellos opcionales, donde un constructor tradicional se vuelve difícil de leer y propenso a errores.

En SecureBallot, una Elección no se define solo con un nombre y una fecha. Requiere candidatos, mesas de votación, fecha de apertura y cierre, si permite voto en blanco, y un umbral de quórum. Una implementación directa mediante un constructor con todos estos parámetros generaría el problema conocido como "constructor telescópico":

```
public Eleccion(nombre, candidatos, mesas, fechaApertura, fechaCierre,
permiteVotoBlanco, umbralQuorum) { ... }
```

Este enfoque hace que el código que crea el objeto sea difícil de leer y aumenta el riesgo de pasar un parámetro en la posición equivocada, especialmente cuando varios parámetros son del mismo tipo de dato.

2. Problema identificado

El problema principal identificado fue la dificultad de construir un objeto con múltiples parámetros opcionales sin recurrir a constructores extensos o sobrecargados. Este problema aumenta a medida que el objeto necesita más configuraciones, dificultando el mantenimiento y la legibilidad del código.

Con Builder se busca separar el proceso de construcción (paso a paso, con métodos nombrados claramente) de la representación final del objeto, de forma que el código cliente no tenga que preocuparse por el orden ni la cantidad de parámetros.

3. Patrón aplicado

Se implementó el patrón Builder para la construcción de la configuración de una Elección.

La estructura utilizada corresponde a la variante clásica del patrón, compuesta por:

- El Producto (Eleccion): representa el objeto final ya construido.
- El Builder abstracto (EleccionBuilder): define los pasos posibles de construcción, sin implementarlos.
- El Concrete Builder (EleccionBuilderEstandar): implementa cada paso, construyendo internamente una instancia de Eleccion pieza por pieza.
- El Director (ConfiguradorEleccionEstandar): conoce el orden correcto en que deben ejecutarse los pasos para construir una elección "estándar", pero delega la construcción real al Builder.

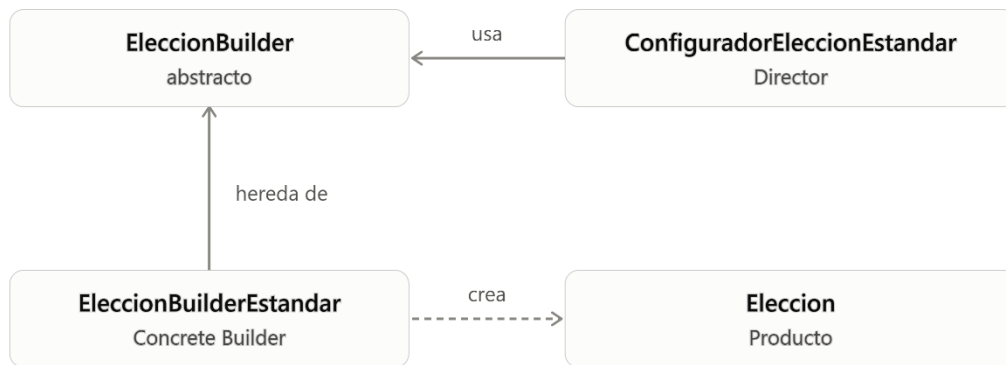


Ilustración 5. Diagrama de clases del patrón Builder aplicado a la configuración de una Elección.

4. Aplicación en SecureBallot

La aplicación del patrón se realizó sobre la configuración de una Elección antes de que esta inicie.

El Director orquesta la construcción llamando, en orden, a los métodos del Builder: `definirNombre()`, `agregarCandidato()` (una vez por cada candidato), `agregarMesa()` (una vez por cada mesa), `definirFechas()`, `permitirVotoBlanco()` y `definirQuorum()`. Al final, `obtenerEleccion()` entrega el objeto completo.

Un punto importante de esta implementación es que el objeto **Eleccion** resultante se integró con el **GeneradorReporte** desarrollado en la semana anterior (Factory Method), sin modificar ni duplicar ese código. Los datos de la elección construidos con Builder alimentan directamente el proceso de generación de reportes ya existente, demostrando que los patrones trabajan en conjunto dentro del proyecto.

5. Relación con SOLID

DIP (Dependency Inversion Principle):

El Director (ConfiguradorEleccionEstandar) depende de la abstracción EleccionBuilder, no de la implementación concreta EleccionBuilderEstandar. Esto se comprobó mediante una prueba automatizada que reemplaza el Concrete Builder por un builder alternativo que cumple la misma interfaz, sin que el Director requiera ningún cambio.

SRP (Single Responsibility Principle):

Cada clase tiene una responsabilidad clara: Eleccion solo representa el dato final, el Builder solo sabe construir paso a paso, y el Director solo conoce el orden de construcción.

OCP (Open/Closed Principle):

Es posible crear un nuevo Concrete Builder (por ejemplo, para una elección con reglas distintas) sin modificar el Director ni el Producto existentes.

6. Evidencia de implementación

La implementación del patrón se encuentra organizada dentro de la carpeta:

- Semana05_Builder/
- Entre los componentes desarrollados se encuentran:
- eleccion.js (Producto)
- eleccion-builder.js (Builder abstracto y Concrete Builder)
- configurador-eleccion.js (Director)
- builder.test.js
- index.js

- demo.js

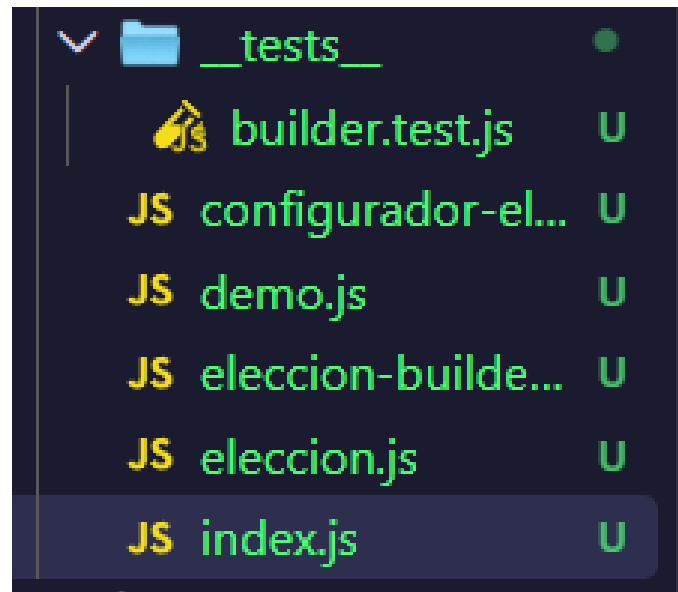


Ilustración 6. Estructura de archivos de Semana05_Builder.

JS eleccion.js U X

Semana05_builder > JS eleccion.js > ...

```
1  /**
2   * BUILDER – Producto.
3   *
4   * Representa una Elección ya construida. No sabe nada sobre CÓMO se
5   * construyó paso a paso – esa responsabilidad vive en el Builder.
6   */
7  class Eleccion {
8    constructor() {
9      this.nombre = null;
10     this.candidatos = [];
11     this.mesas = [];
12     this.fechaApertura = null;
13     this.fechaCierre = null;
14     this.permiteVotoBlanco = false;
15     this.umbralQuorum = 0;
16   }
17 }
18
19 module.exports = { Eleccion };
```

JS eleccion-builder.js U X

Semana05_builder > JS eleccion-builder.js > ...

```
1  /**
2   * BUILDER – Builder abstracto y Concrete Builder.
3   *
4   * El Builder abstracto declara los pasos posibles de construcción.
5   * El Concrete Builder los implementa y va armando una instancia real
6   * de Eleccion internamente, pieza por pieza.
7   */
8  const { Eleccion } = require('./eleccion');
9
10 // [INGREDIENTE] BUILDER (abstracto)
11 class EleccionBuilder {
12   definirNombre(nombre) { throw new Error('definirNombre() debe implementarse'); }
13   agregarCandidato(candidato) { throw new Error('agregarCandidato() debe implementarse'); }
14   agregarMesa(mesa) { throw new Error('agregarMesa() debe implementarse'); }
15   definirFechas(apertura, cierre) { throw new Error('definirFechas() debe implementarse'); }
16   permitirVotoBlanco(permite) { throw new Error('permitirVotoBlanco() debe implementarse'); }
17   definirQuorum(umbral) { throw new Error('definirQuorum() debe implementarse'); }
18   obtenerEleccion() { throw new Error('obtenerEleccion() debe implementarse'); }
19 }
20
21 // [INGREDIENTE] CONCRETE BUILDER
22 class EleccionBuilderEstandar extends EleccionBuilder {
23   constructor() {
24     super();
25     this.eleccion = new Eleccion();
26   }
27
28   definirNombre(nombre) {
29     this.eleccion.nombre = nombre;
30     return this;
31   }
32
33   agregarCandidato(candidato) {
34     this.eleccion.candidatos.push(candidato);
35     return this;
36   }
37 }
```

```

38     agregarMesa(mesa) {
39         this.eleccion.mesas.push(mesa);
40         return this;
41     }
42
43     definirFechas(apertura, cierre) {
44         this.eleccion.fechaApertura = apertura;
45         this.eleccion.fechaCierre = cierre;
46         return this;
47     }
48
49     permitirVotoBlanco(permite) {
50         this.eleccion.permiteVotoBlanco = permite;
51         return this;
52     }
53
54     definirQuorum(umbral) {
55         this.eleccion.umbralQuorum = umbral;
56         return this;
57     }
58
59     obtenerEleccion() {
60         return this.eleccion;
61     }
62 }
63
64 module.exports = { EleccionBuilder, EleccionBuilderEstandar };

```

Ilustración 7. Implementación del Producto y el Builder (EleccionBuilder / EleccionBuilderEstandar).

```
JS configurador-eleccion.js U X
Semana05_builder > JS configurador-eleccion.js > ConfiguradorEleccionEstandar

1  /**
2   * BUILDER – Director.
3   *
4   * Conoce el ORDEN correcto de los pasos de construcción para un caso
5   * de uso concreto (una elección "estándar"), pero no sabe construir
6   * nada por sí mismo – delega cada paso al Builder que recibe.
7   *
8   * Nota de diseño: este Director no depende de EleccionBuilderEstandar
9   * directamente, sino de cualquier objeto que cumpla la interfaz
10  * EleccionBuilder. Así, si mañana se necesita un builder distinto
11  * (por ejemplo uno para pruebas), este Director lo puede usar sin
12  * cambiar ni una línea.
13  */
14  class ConfiguradorEleccionEstandar {
15      configurar(builder, datos) {
16          builder.definirNombre(datos.nombre);
17          datos.candidatos.forEach((c) => builder.agregarCandidato(c));
18          datos.mesas.forEach((m) => builder.agregarMesa(m));
19          builder.definirFechas(datos.fechaApertura, datos.fechaCierre);
20          builder.permiteVotoBlanco(datos.permiteVotoBlanco);
21          builder.definirQuorum(datos.umbralQuorum);
22          return builder.obtenerEleccion();
23      }
24  }
25
26  module.exports = { ConfiguradorEleccionEstandar };
```

Ilustración 8. Implementación del Director (ConfiguradorEleccionEstandar).

7. Evidencia de pruebas

Para verificar el funcionamiento de la implementación se utilizaron pruebas automatizadas mediante Jest.

Se ejecutó el siguiente comando:

```
npx jest
```

Las pruebas verificaron:

Que el Builder abstracto bloquea su uso directo si un método no fue implementado.

Que el Concrete Builder construye correctamente una Eleccion con todos los datos esperados.

Que cada método del builder retorna la propia instancia, permitiendo el encadenamiento de llamadas.

Que el Director arma correctamente una Eleccion completa a partir de un objeto de datos.

Que el Director funciona con cualquier builder que cumpla la interfaz esperada, sin depender de una implementación concreta (evidencia práctica de DIP).

En total se aprobaron 5 pruebas de 5, sin pruebas fallidas, sumándose a las pruebas de Singleton y Factory Method desarrolladas en semanas anteriores.

```

PS C:\Users\johin\SecureBallot> npx jest
PASS Semana05_builder/___tests___/builder.test.js
PASS Semana04_PFM/factory-method.test.js
  ● Console

    console.log
      [2026-09-12T19:20:39.616Z] [INFO] Reporte de resultados electorales generado
      at LoggerCentralizado.log [as registrar] (Semana03/singleton-votacion.js:162:13)

    console.log
      [2026-09-12T19:20:39.621Z] [INFO] Reporte de resultados electorales generado
      at LoggerCentralizado.log [as registrar] (Semana03/singleton-votacion.js:162:13)

    console.log
      [2026-09-12T19:20:39.621Z] [INFO] Reporte de resultados electorales generado
      at LoggerCentralizado.log [as registrar] (Semana03/singleton-votacion.js:162:13)

    console.log
      [2026-09-12T19:20:39.623Z] [INFO] Reporte de resultados electorales generado
      at LoggerCentralizado.log [as registrar] (Semana03/singleton-votacion.js:162:13)

    console.log
      [2026-09-12T19:20:39.626Z] [INFO] Reporte de resultados electorales generado
      at LoggerCentralizado.log [as registrar] (Semana03/singleton-votacion.js:162:13)

PASS Semana03/___tests___/singleton.test.js
  ● Console

    console.log
      [2026-09-12T19:20:39.760Z] [AUDITORIA_VOTO] Voto registrado
      at LoggerCentralizado.log [as registrar] (Semana03/singleton-votacion.js:162:13)

    console.log
      [2026-09-12T19:20:39.765Z] [DEBUG] Conexión adquirida (1/100)
      at LoggerCentralizado.log [as registrar] (Semana03/singleton-votacion.js:162:13)

Test Suites: 3 passed, 3 total
Tests:       21 passed, 21 total
Snapshots:   0 total
Time:        5.884 s
Ran all test suites.
PS C:\Users\johin\SecureBallot>
PS C:\Users\johin\SecureBallot>

```

Ilustración 5. Ejecución de pruebas del patrón Builder con Jest.

8. Aporte del patrón al proyecto

La implementación de Builder permite que SecureBallot construya objetos de configuración complejos de forma clara y ordenada, sin recurrir a constructores con múltiples parámetros posicionales.

Uno de los principales aportes es la legibilidad: el proceso de construcción se lee como una secuencia de pasos con nombres descriptivos, en vez de una lista de valores sin contexto. Además, al separar el Director del Builder, es posible introducir nuevas formas de construir una Elección sin modificar el código existente.

- De esta manera, el patrón contribuye a:
- Evitar el problema del constructor telescópico.
- Separar el proceso de construcción de la representación final.
- Facilitar la lectura y el mantenimiento del código de configuración.
- Reforzar el principio de Inversión de Dependencias entre el Director y el Builder.
- Integrar la configuración de una Elección con el Factory Method desarrollado previamente.

9. Resultado de la semana

Al finalizar la semana se logró implementar y probar el patrón Builder dentro de SecureBallot, utilizando como caso de aplicación la construcción de la configuración de una Elección.

La implementación permitió demostrar la diferencia entre construir un objeto mediante un constructor con múltiples parámetros y construirlo paso a paso mediante un proceso guiado por un Director.

La ejecución de las pruebas automatizadas confirmó el correcto funcionamiento de la implementación, obteniendo 5 pruebas aprobadas de 5, y se comprobó la integración funcional entre Builder y el Factory Method desarrollado en la semana anterior.

```
PS C:\Users\johin\SecureBallot> cd Semana05_builder
PS C:\Users\johin\SecureBallot\Semana05_builder> node demo.js
--- Elección construida (import via index.js) ---
Eleccion {
  nombre: 'Representante Estudiantil 2026',
  candidatos: [ 'Ana Torres', 'Luis Gómez' ],
  mesas: [ 'Mesa 1', 'Mesa 2' ],
  fechaApertura: '2026-09-04T08:00',
  fechaCierre: '2026-09-04T17:00',
  permiteVotoBlanco: true,
  umbralQuorum: 500
}
```

```
--- Reporte via Factory Method existente ---
[2026-09-12T19:25:40.864Z] [INFO] Reporte de resultados electorales generado
ACTA DE ESCRUTINIO (formato PDF - representación textual)
Elección: Representante Estudiantil 2026
Fecha: 2026-09-04T17:00
-----
Ana Torres: 340 votos
Luis Gómez: 298 votos
-----
Total de votos: 638
PS C:\Users\johin\SecureBallot\Semana05_builder> |
```

Ilustración 6. Ejecución de demo.js mostrando la Elección construida con Builder y el reporte generado con Factory Method.